NETWORK APPLICATION PROJECT

# DISTRIBUTED AND STREAMING BASE CLIENT-SERVER COLLABORATION APPLICATION

**Course:** Network Application Development

**Duration:** 6 Weeks (Take-home Project)

**Project Type:** Individual / Pair Project

## I. INTRODUCTION

### 1. CURRENT PRACTICES

With the rapid global shift toward remote work, distributed teams, and decentralized systems, reliance on robust, real-time network applications has never been higher. Modern organizations require seamless data exchange over local and wide-area networks, while navigating challenges like latency, packet loss, and data security. Understanding how to build resilient network software from the ground up is a critical skill for today's software and network engineers.

### 2. THE NEED FOR THIS APPLICATION

While commercial solutions (like Slack, Microsoft Teams, or cloud storage) exist, they often route data through centralized third-party servers, which can raise privacy, compliance, and bandwidth concerns for closed-network environments. There is a distinct need for a lightweight, secure, and self-hosted network application that allows users on the same network (or VPN) to communicate and share data directly without relying on external corporate infrastructure.

### 3. GOAL OF THE APPLICATION

The goal of this project is to build a **Distributed and Streaming-based Client-Server Collaboration Application**. Students will design and implement a custom application layer protocol over UDP STREAMING that enables real-time messaging, active user discovery, and reliable file transfer between multiple clients over a network. Due to the native requirement of messaging application, **reliable communication** must be provided.

**4. Features of the Application**

- **User Authentication & Discovery:** Users can log in, broadcast their presence, and see a live list of other active peers on the network.

- **Streaming-based Messaging:** Support for both broadcast (global chat) and unicast (private direct messaging). Refer **Steam GameNetworkingSocket** and **Google QUIC** for message-oriented transport layer.
- **Reliable File Transfer:** The ability to send and receive files of various formats and sizes without data corruption.
- **Connection Resilience:** The app gracefully handles sudden client disconnects, timeouts, and network interruptions.
- **Secure Data Transmission:** Basic encryption applied to the payloads to prevent packet-sniffing vulnerabilities.

**5. Target Users**

- Internal corporate teams operating on secure, air-gapped, or isolated local area networks (LAN).
- Students or study groups needing a fast, peer-to-peer way to collaborate and share files without internet access.
- Privacy-conscious individuals who prefer self-hosted communication tools.

**6. Techniques and Technologies Integrated**

- **Socket Programming:** Implementation of TCP (for reliable file/message transfer) and UDP (for heartbeat/discovery broadcasts).
- **Concurrency:** Multi-threading or Asynchronous I/O to handle multiple simultaneous client connections without blocking the main thread.
- **Application-Layer Protocol Design:** Designing custom packet structures (Headers, Payloads, End-of-Message flags).
- **Sequencing and ordering**: Maintaining monotonic sequence number with window-based sliding window for flow control and in-order delivery.
- **Acknowledgement and Retransmission:** Efficient retransmitting of only missing segments.
- **Congestion control:** Support streaming specific with BBR / CUBIC streaming support.
- **Cryptography:** Integration of TLS/SSL sockets or symmetric encryption (e.g., AES) for data payloads.
- **Serialization:** Encoding and decoding structured data (JSON, XML, or custom binary packing) over the wire.

**7. Design Requirements**

- **Architecture:** Must clearly separate network logic, data processing, and the user interface.
- **Time-sensitive streaming**: The system must provide an async mechanism to fetch data from a stream without any. The streaming only handles chunk of bytes in UDP and leaves your protocol logics in application layer in which you eliminate Head-of-line (HoL) blocking.
- **User Interface:** Can be a clean Command Line Interface (CLI) or a Graphical User Interface (GUI). It must be intuitive and display network status clearly.

- **Scalability:** The server/tracker must be capable of handling at least 10-20 concurrent clients efficiently.

**8. Technical Requirements**

- **Programming Language:** Python, Java, C++, or Go (Instructor to confirm restricted languages, if any).
- **No High-Level Networking Libraries:** Students must use raw sockets (socket library in Python, java.net.Socket in Java, etc.). HTTP frameworks (like Flask/Django) or WebSockets are strictly prohibited to ensure core networking concepts are practiced.
- **Environment:** Must run successfully on standard operating systems (Windows, macOS, or Linux) using localhost or local IP addressing.

**9. Development Process & Grading Rubric (10 Points Total)**

This project is broken down into a 10-step development lifecycle. Each successfully implemented step is worth **1 point**.

| Step | Milestone | Score |
|---|---|---|
| 1 | **Setup & Protocol Design:** Define the custom packet structure, message types (e.g., AUTH, MSG, FILE), and state diagrams in documentation before coding. | 1 pt |
| 2 | **Basic Client-Server Handshake:** Implement basic TCP socket connection where a client can connect to the server and receive an acknowledgment. | 1 pt |
| 3 | **Authentication & State Management:** Implement a login mechanism. The server must track connected users, their IP addresses, and active ports. | 1 pt |
| 4 | **Concurrency Implementation:** Integrate multi-threading or **Async** I/O so the server can handle multiple clients connecting and disconnecting simultaneously. | 1 pt |
| 5 | **Time-sensitive streaming:** Implement the routing logic for clients to send messages to the server, and the server to broadcast or forward them to the correct recipient. | 1 pt |
| 6 | **Reliable File Transfer:** Implement file serialization, chunking (for large files), and reassembly over UDP without data corruption. | 1 pt |

| Step | Milestone | Score |
|---|---|---|
| 7 | **Network Resilience & Error Handling:** Implement try/except blocks to handle socket timeouts, broken pipes, and graceful client exits without crashing the server. | 1 pt |
| 8 | **Security Layer:** Encrypt the data payloads (using a standard cryptographic library) before sending, and decrypt upon receipt. | 1 pt |
| 9 | **HTTP based UI with client-side JavaScript only:** Build a user-friendly interface that clearly displays incoming messages, file transfer progress bars, and active user lists. | 1 pt |
| 10 | **Testing & Code Quality:** Code must be cleanly formatted, commented, and pass local network simulation tests (connecting from multiple terminal windows). | 1 pt |
| | **TOTAL** | **10 pts** |

**10. Submission Guidelines**

**Suggested Deadline:**

- Sunday, November 17, 2026, at 11:59 PM.

**Required Submission Content:**

Students must submit a .zip file or a link to a private GitHub repository containing the following:

1. **Source Code:**
    - All client and server scripts.
    - A README.md file with strict, step-by-step instructions on how to install dependencies, run the server, and connect the clients.
2. **Technical Explanation Report (PDF):**
    - **System Architecture:** A high-level diagram showing how clients and the server interact.
    - **Protocol Specification:** A breakdown of the custom packet/message structures designed for the application.
    - **Sequence Diagram:** A flow chart showing the lifecycle of a message and a file transfer.

- **Challenges & Solutions:** A brief 1-page reflection on network challenges faced (e.g., resolving deadlocks, handling packet fragmentation) and how they were solved.